Hyper Twin Phase 1 Prototype

Core ECS Design (The "Brain")

Entities (The "Things")

We do not use classes. We use **Components**.

- **Robot:** `[Position, Velocity, RobotState, CargoCapacity, Battery]`
- **Shelf:** `[Position, StorageContainer, ZoneID]`
- **Package:** `[Position, SKU_ID, Destination_ID]`

Systems (The "Logic")

These run in parallel on the CPU.

1. `ingest_sensor_data` : Reads CSV rows and spawns entities (Simulating IoT connection).
2. `pathfinding_system` : Uses Flow Fields (Dijkstra) to guide robots. *Why Flow Fields? Because A* is too slow for 10k units.*
3. `collision_avoidance` : Simple ray-casting to stop robots crashing.
4. `metrics_system` : Calculates "Throughput per Hour" and updates the UI Resource.

Phase 1 Goal (First 48 Hours)

Deliverable: A running window showing a gray floor, a "Robot" (red cube), and a camera. You can move the robot using WASD keys.

1. Environment Setup

- [x] ~~Install Rust Toolchain (rustup).~~
- [x] ~~Install VS Code & Configure Extensions (`rust-analyzer`).~~
- [x] ~~Initialize Project (`cargo new hyper_twin_rs`).~~
- [x] ~~Configure `Cargo.toml` with Bevy dependencies.~~
- [x] ~~**Milestone:** Run `cargo run` and see "Hello World" in the terminal.~~

2. The ECS Skeleton (The "Bevy Way")

- [x] ~~Create `components.rs` : Define `Robot` and `Velocity` structs.~~
- [x] ~~Create `main.rs` : Set up the Bevy `App` builder.~~
- [x] ~~Add `DefaultPlugins` (Window, Renderer, Input).~~
- [x] ~~**Milestone:** Run the app and see an empty gray window.~~

3. The World Builder

- [x] ~~Create a `setup_scene` system.~~
- [x] ~~Spawn a 3D Camera (`Camera3dBundle`).~~
- [x] ~~Spawn a Light source (`PointLightBundle`).~~
- [x] ~~Spawn a Floor (`PbrBundle` with a Plane mesh).~~
- [x] ~~**Milestone:** See a lit floor plane in the window.~~

4. The Agent (Robot)

- [x] ~~Create a `spawn_robot` system.~~
- [x] ~~Spawn a Red Cube (`PbrBundle` + `Robot` Component).~~
- [x] ~~**Milestone:** A wild Red Cube appears on the floor.~~

5. Input & Movement Logic

- [x] ~~Create a `move_robot` system.~~
- [x] ~~Query for `KeyboardInput` .~~
- [x] ~~Query for `(&mut Transform, &Robot)` .~~
- [x] ~~Apply velocity based on key presses.~~
- [x] ~~**Milestone:** WASD moves the cube smoothly.~~